

<https://youtu.be/k3ZoNJH1Xiw>

Video

Introduction

Hello, we are Group 12 Swarming Agents. This class is part of the step-based simulation system, and its main job is to decide a movement for the agent at each tick.

The Swarming Agent works with the agent data and the world, but it does not directly control the simulation itself. Instead, it prepares actions that the world can process. It reads the agent's current state, such as its position, destination, and activity status, and then chooses a movement based on that information. Since the module is templated, each Swarming Agent can make decisions that are tailored to the specific world it is working in.

The world owns the agent, checks the steps it produces, and applies the results according to the simulation rules.

So overall, the Swarming Agent is the part that handles movement decisions, while the world manages how those decisions affect the simulation.

Step Logic

I'll just go over a quick overview of our GetTurn() structure, basically our get turn checks at compile time which data it uses whether it is traffic data or Infection data. From there we run the specific getturn for that world.

Looking at our traffic get turn our logic consists of firstly checking if our data is active, from there it checks if there is a destination set for the agent to swarm to. If there is not a destination set it creates a movement step to swarm randomly and adds it to the stepcontainer and returns. If there is a set destination for the agent to swarm to the logic checks if the agent is at the destination, if it is it returns the empty container, implying do nothing. If the agent is not at the destination it then chooses the closest option to get to the destination.

From there we create 4 movement steps (each in every direction) going from best to worst move adding it to the step container and returning it to the world, for the world to decide if our move is "valid" and then processing our movement.

Looking at the infection get turn, we have similar functionality, but the infection world simply swarms randomly, so the only movement step being created is one to swarm randomly. We then return that container with the random movement back to the infection world to complete our move.

Movement

A SwarmingAgent keeps count of how many times it visited every cell on the grid, and it keeps track of the last six positions it has been to. This is what prevents it from getting stuck in loops.

It then checks if it has a set destination. If it's in the infection world, it doesn't have a destination and it just picks a random direction with a preference for cells that it hasn't been to recently. If it is in the traffic world, then it does have a destination. If it's already there, it does nothing. But if it isn't, we use a scoring system:

Each direction (up, down, left, right) is given a score that is calculated based on the Manhattan distance from the current location to the destination plus a visit penalty. A lower score means a better move. There is also a small random tiebreaker to prevent agents from all making identical choices. The four options are then sorted by score. If the best choice is blocked, it can automatically go to the next best choice.

The visit penalty is really important to the whole design. It prevents agents from taking the shortest path and pile up behind walls. It allows agents that are passing through the same cells to use alternate paths, which naturally spreads out the traffic and avoids congestion.

Challenges

One of the biggest challenges we faced was agents getting stuck in loops. Early in development, we noticed that agents would repeatedly move between the same few positions without making progress toward their destination.

Our first attempt to fix this was to completely block any repeat movements. This worked for very simple paths, but as the environment became more complex, it started to fail. The agent would run out of valid moves and begin repeating the same loops.

Next, we tried a more advanced approach where the agent remembered all previously visited paths and even walls. We assigned penalties to those nodes to discourage the agent from choosing them again. While this helped reduce looping, it introduced a new issue. If an agent was temporarily blocked by another agent, it would treat that path like a wall and avoid it in the future, which caused strange and inefficient behavior.

The final solution was to change how movement decisions were returned. Instead of trying to block or heavily penalize paths, the agent now returns all four possible movement options every tick, ranked by desirability.

This also solved another major issue we were seeing. Previously, agents would sometimes move very slowly when heading in the opposite direction of its goal. This happened because the agent would only return one or two movement options, and if both were blocked, it wouldn't move at all for that tick. By returning all four options, the world can fall back to the next-best option, so the agent continues moving every tick.

Test Cases

The test suite for the `SwarmingAgent` class validates the agent's behavior across both of its supported data types, `TrafficData` and `DiseaseData`, ensuring correctness from basic construction through complex movement logic. The foundational tests confirm that agents store and return their IDs, that initial state can be retrieved without corruption, and that mutator methods like `SetGoal` and `SetActive` properly modify the agent's internal data through the `GetState/SetState` interface. These may seem trivial, but they guard the copy-based state management pattern used by `StepAgentBase` – if that round-trip ever broke silently, downstream world logic would operate on stale data.

The bulk of the suite focuses on the traffic agent's movement policy under different conditions. An inactive agent must produce no steps at

all, an agent that has already reached its destination must also stay still, and an agent with no destination should wander randomly to exactly one cardinal neighbor per turn. When a destination is set, the tests verify that the agent emits all four cardinal directions as ranked fallback options, that the most direct direction toward the goal appears first, and that all four options are unique. These checks validate the scoring function at the heart of the swarming logic – the Manhattan-distance term, the sort order, and the tie-breaking jitter – and they ensure the world's collision-resolution system can safely consume the full list of alternatives without re-querying the agent.

Finally, the suite covers disease-agent movement and the anti-looping mechanism. Three separate tests confirm that DiseaseData agents move to a random cardinal neighbor regardless of whether they are susceptible, infected, or recovered, documenting the design decision that health-state transitions are the world's responsibility rather than the agent's. The anti-looping test runs thirty turns from a fixed position and asserts that the agent visits more than one distinct neighbor, proving that the recent-position history biases the random walk away from cells it has just occupied. Together, these tests provide confidence that the agent behaves correctly in isolation so that integration with the world layer can be tested independently.